

Lab 6: Set Up Nexus Repository and Configure User Access



Name : Eman Khalid

Reg no : 23-BCS-026

Course Code: CS-649

Section: A

GitHub username: EmanKhalid27

Submitted To: Sir Waqas Saleem

Department: Computer Science

1. Introduction

This lab demonstrates how to set up a self-hosted Nexus Repository Manager using Docker on an Ubuntu server. It covers initial setup, user access management, publishing artifacts from Gradle and Maven projects, inspecting repositories via the REST API, managing blob stores, creating a Docker hosted repository, and implementing cleanup policies with blob store compaction.

2. Task Evidence

Task 1: Run Nexus Repository and Complete the Initial Setup

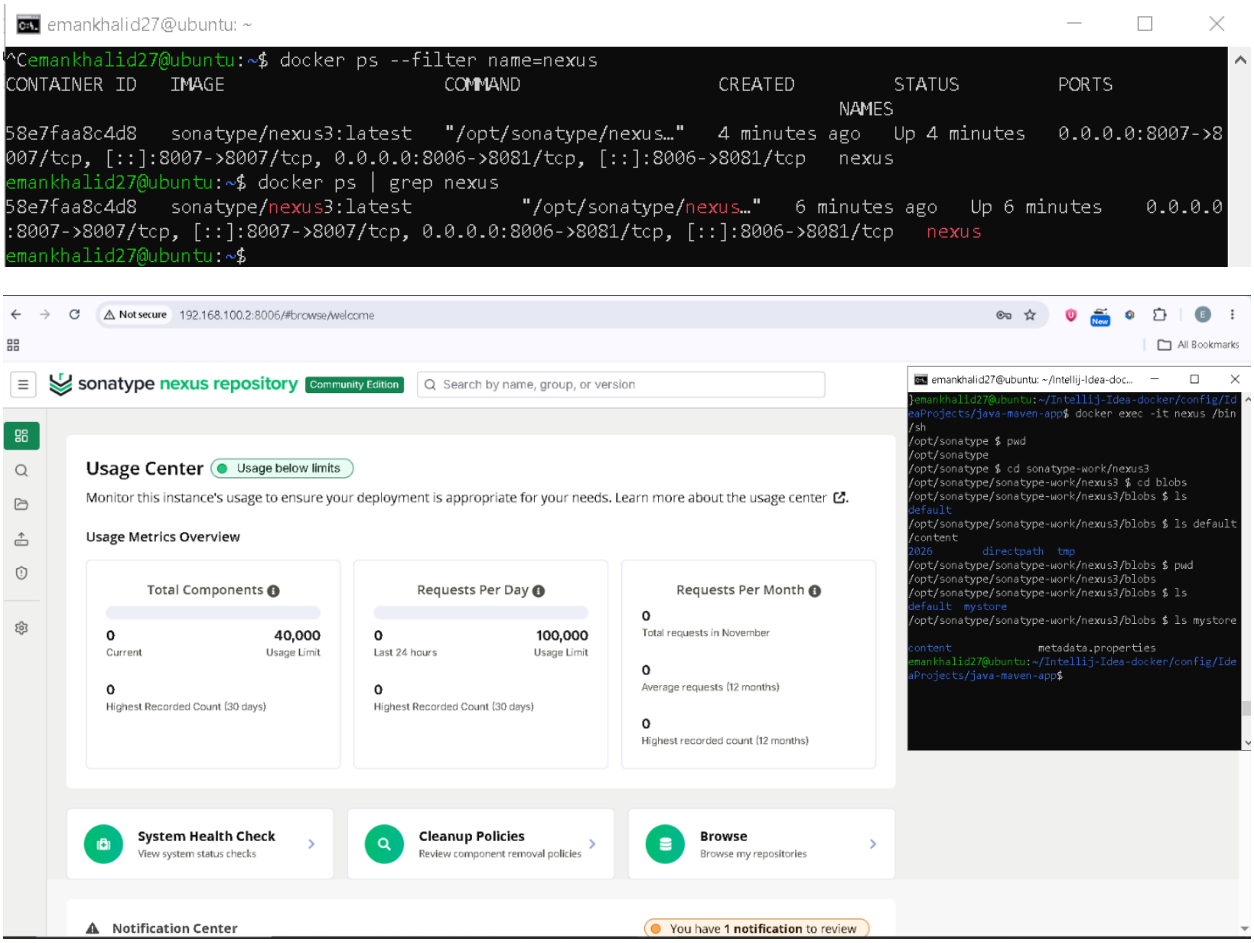


Figure 1.1: The nexus container running with ports 8006 and 8007 published. (nexus_container_running.png)

Task 2: Create a Role and User and Verify Access



Figure 2.1: Verifying the nexus user's access via the Status API returning HTTP 200 OK. (nexus_user_access.png)

Task 3: Build and Publish the Gradle Application to Nexus

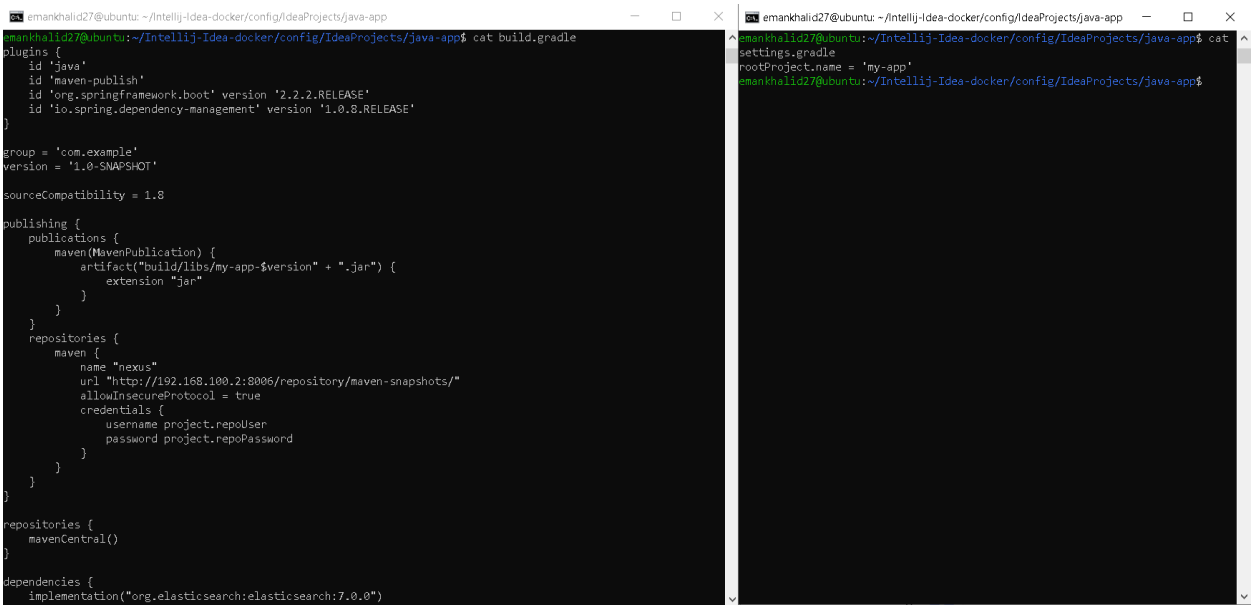


Figure 3.1: Configuration of build.gradle and settings.gradle for Nexus publication. (gradle_nexus_configuration.png)

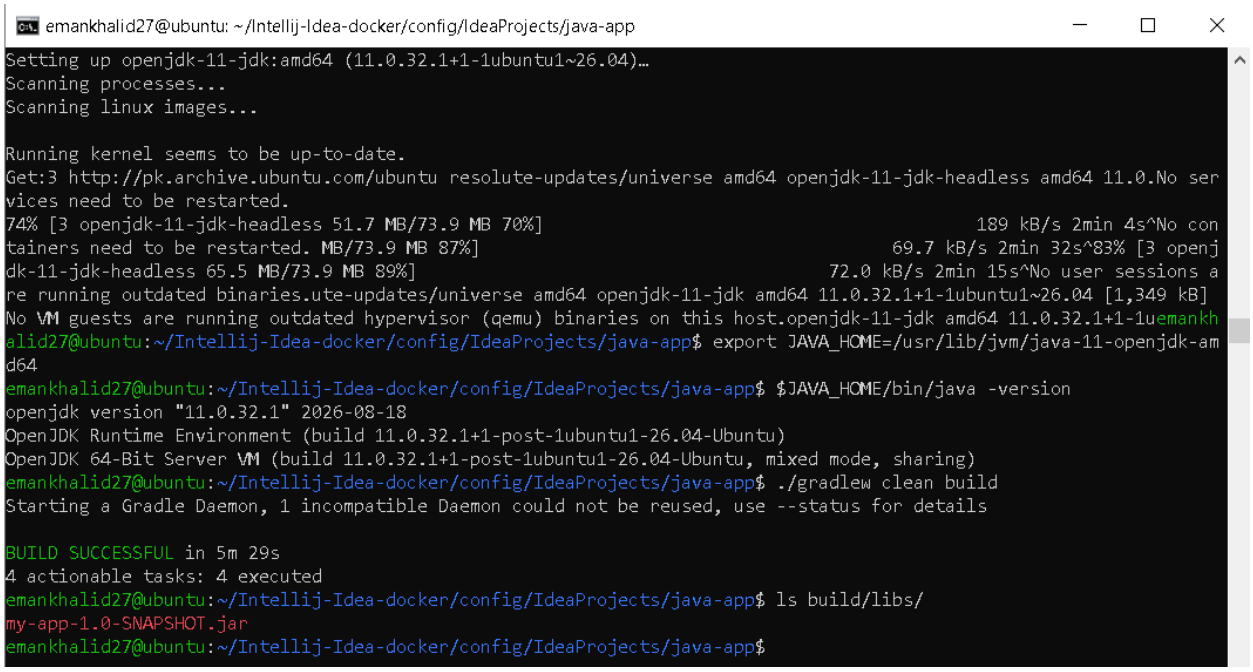


Figure 3.2: Successful Gradle build creating my-app-1.0-SNAPSHOT.jar. (gradle_nexus_build.png)

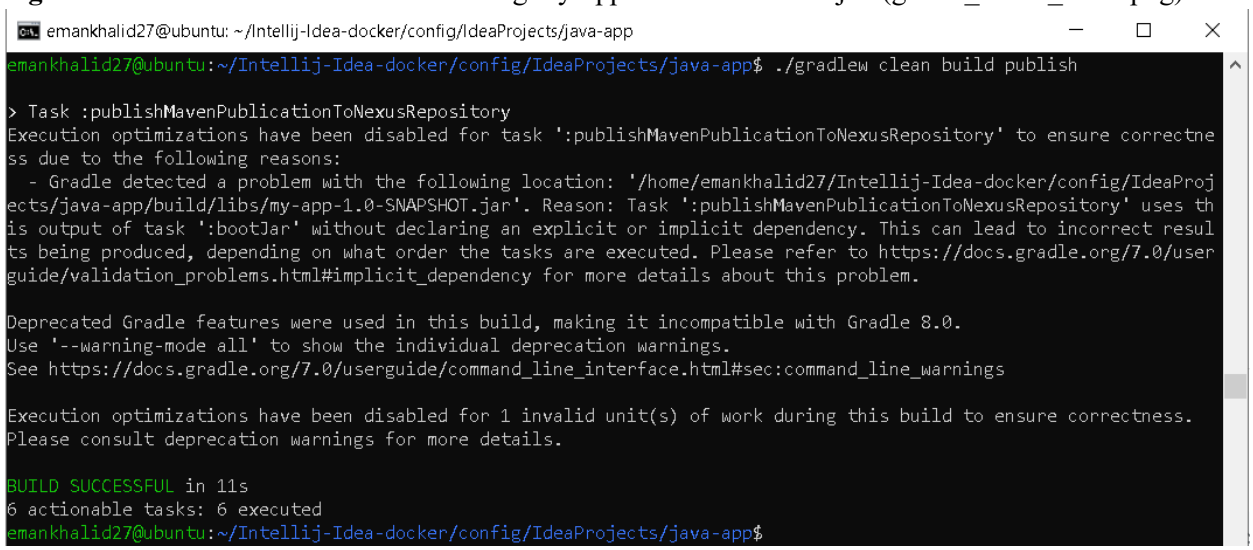


Figure 3.3: Successful Gradle publish to Nexus. (gradle_nexus_publish.png)

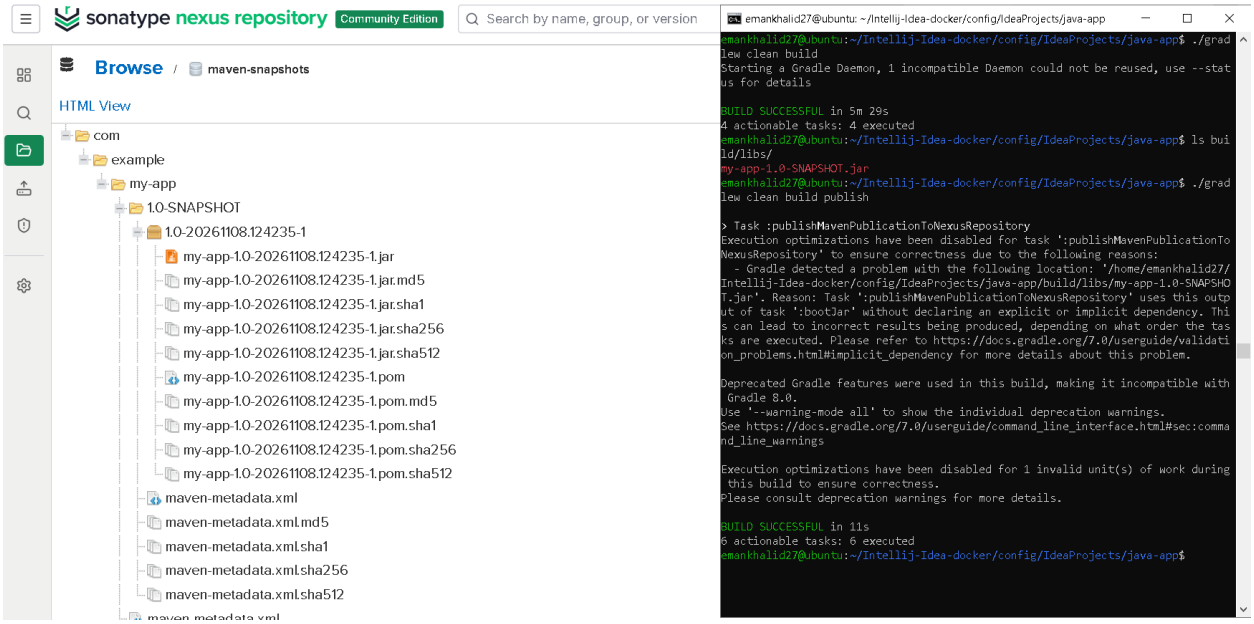


Figure 3.4: The published Gradle artifact visible in the Nexus maven-snapshots repository. (nexus_gradle_artifact.png)

Task 4: Publish the Maven Application to Nexus

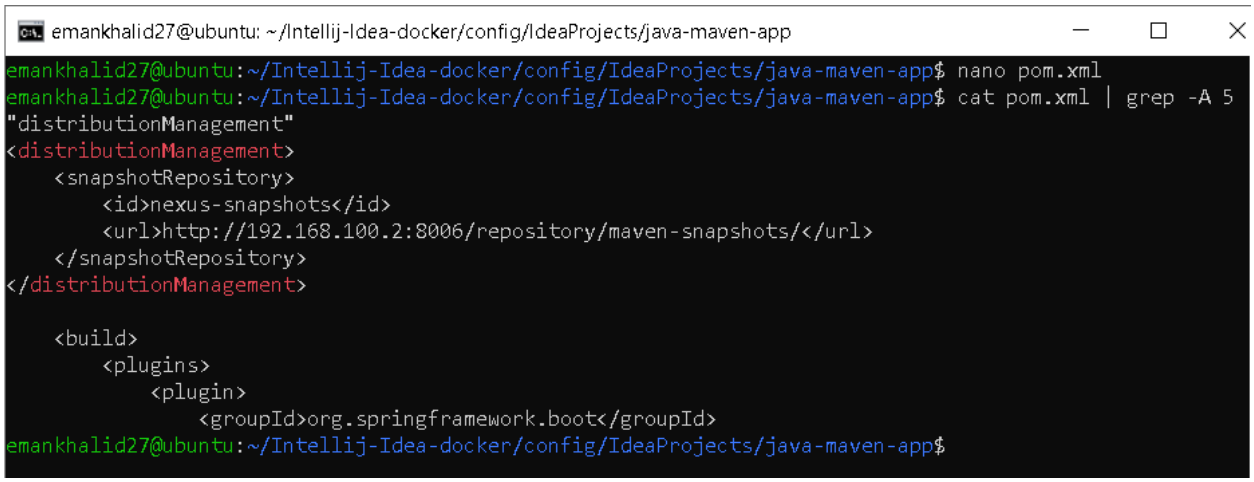


Figure 4.1: The java-maven-app project opened in IntelliJ. (maven_nexus_project.png)

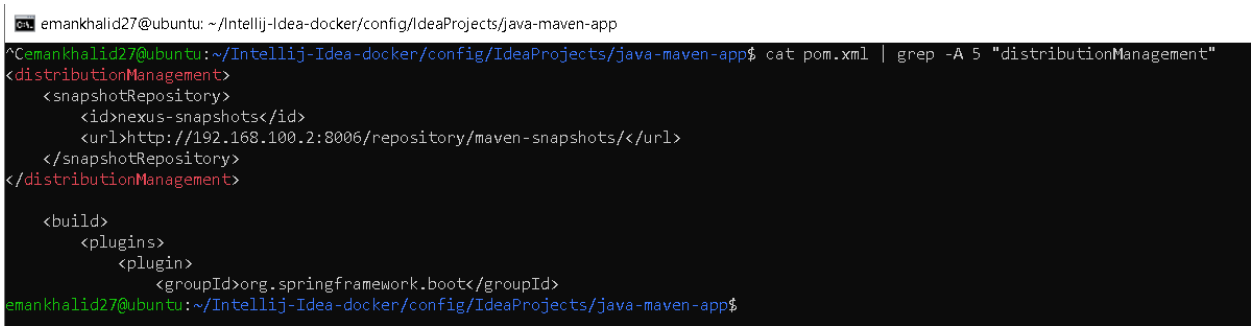


Figure 4.2: The distributionManagement block added to pom.xml. (maven_distribution_management.png)

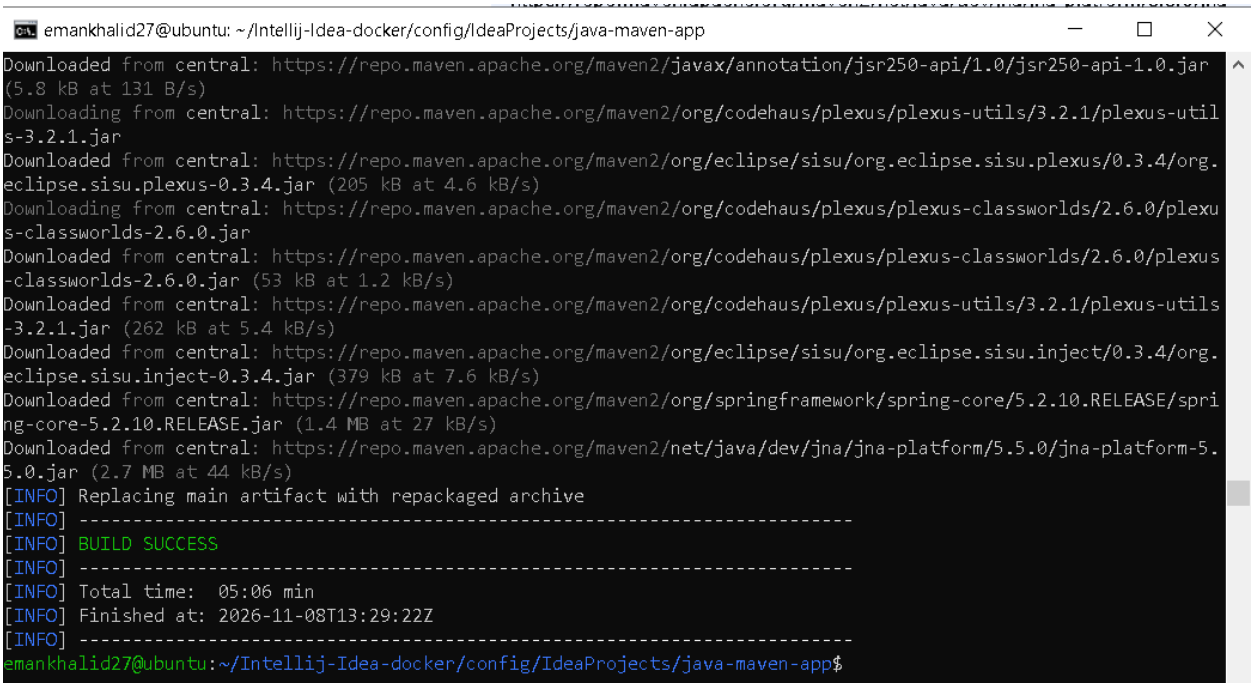


Figure 4.3: Successful mvn package build. (maven_package_nexus.png)

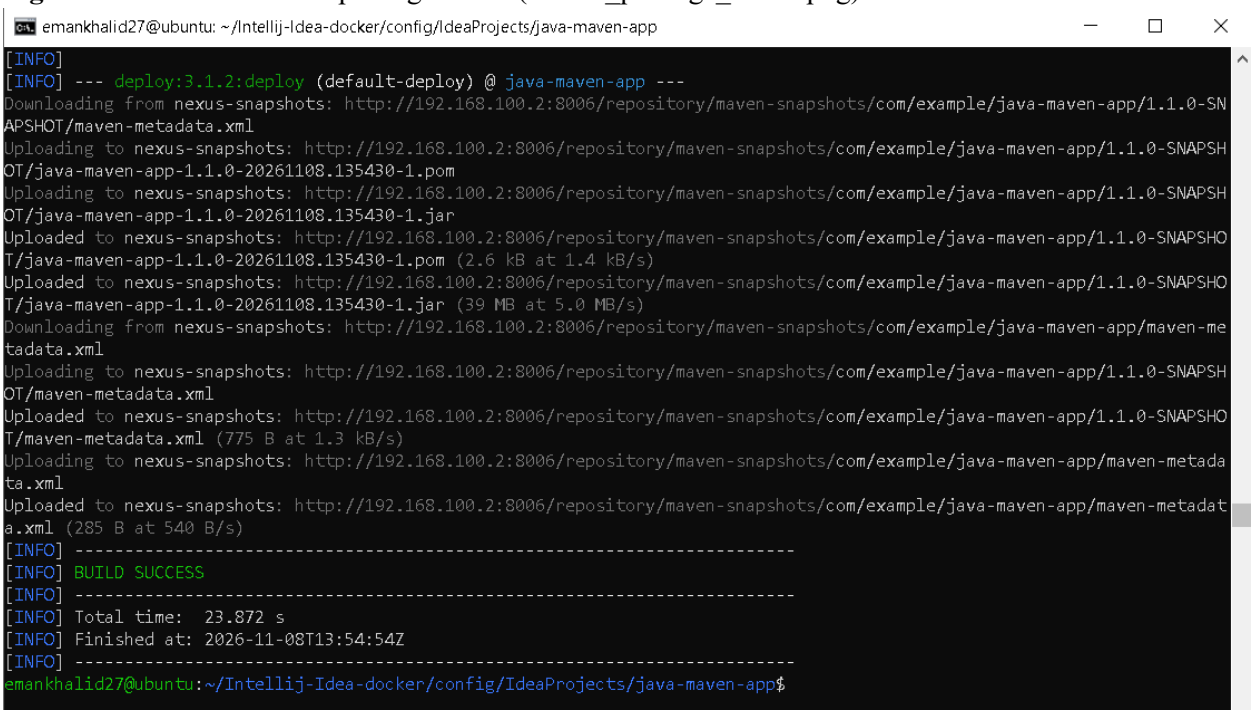


Figure 4.4: Successful mvn deploy to Nexus. (maven_deploy_nexus.png)

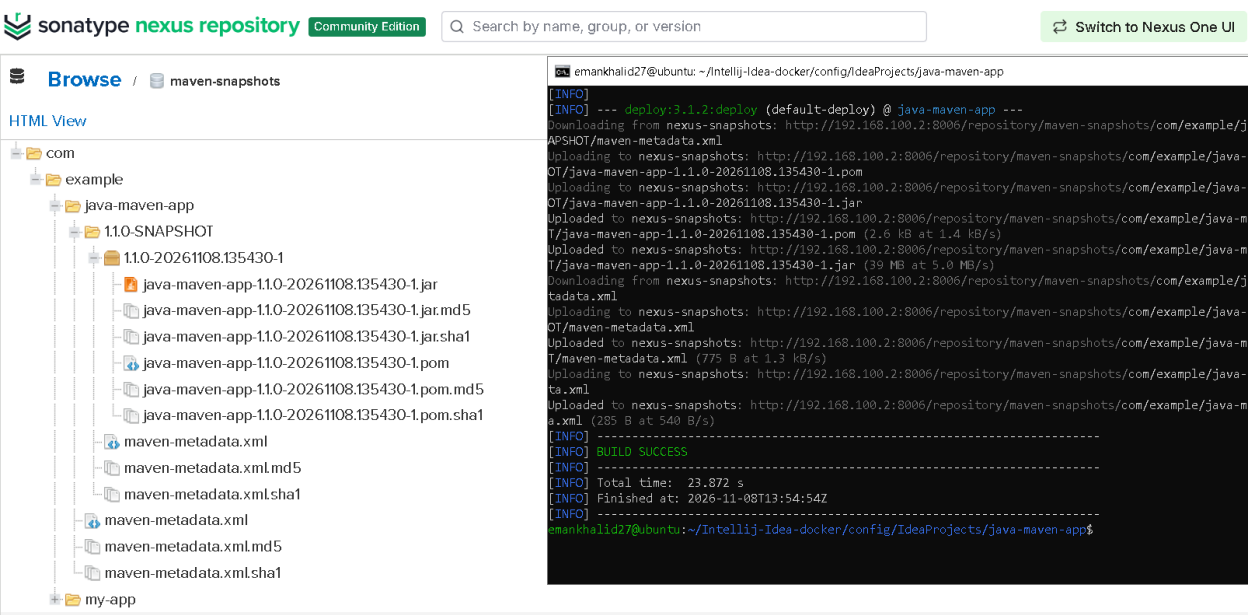


Figure 4.5: The published Maven artifact visible in the Nexus maven-snapshots repository. (nexus_maven_artifact.png)

Task 5: Inspect Nexus Repositories and Components Using the REST API

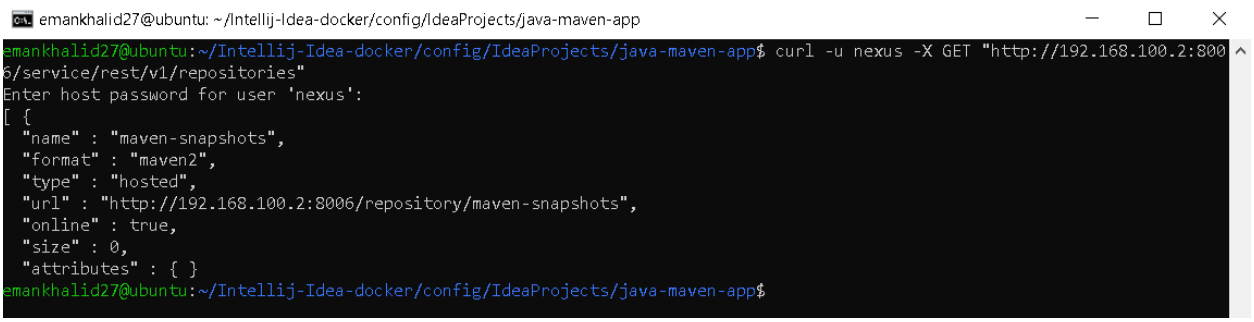


Figure 5.1: Listing repositories as the restricted nexus user. (nexus_repositories_user.png)



Figure 5.2: Listing repositories as the admin user. (nexus_repositories_admin.png)

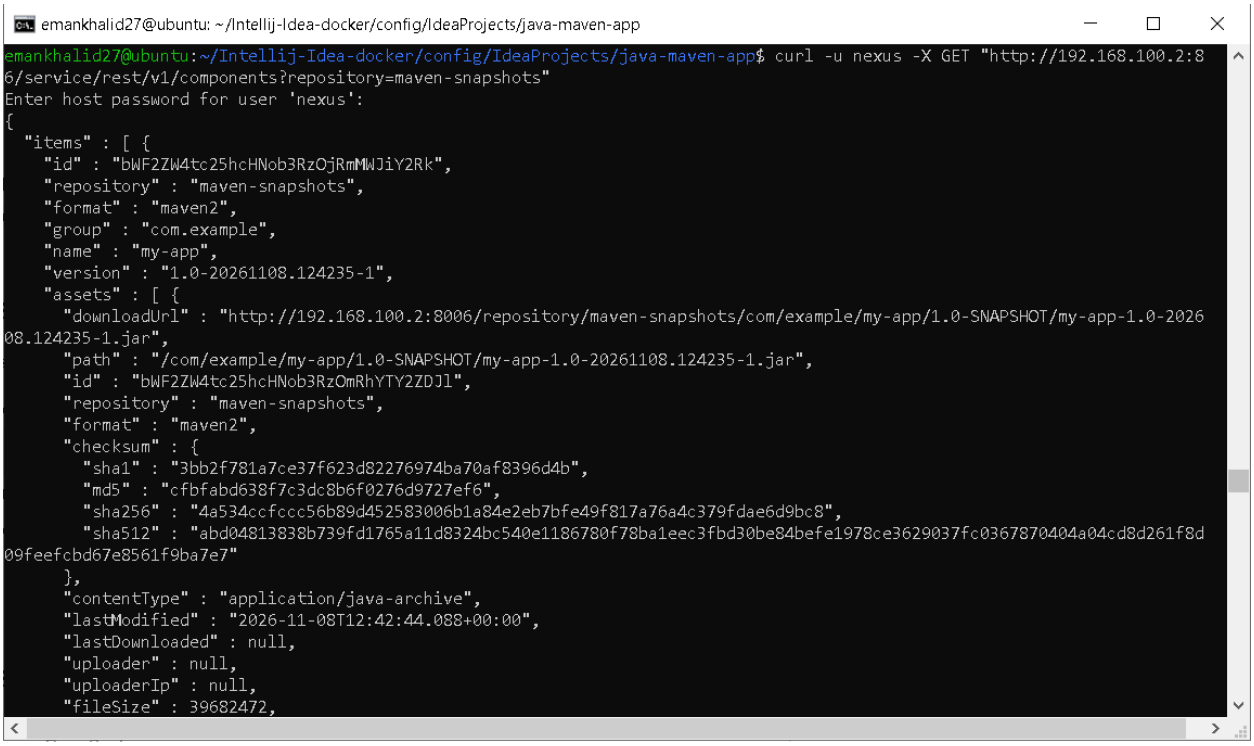


Figure 5.3: Listing components in maven-snapshots and identifying a component ID. (nexus_components_list.png)

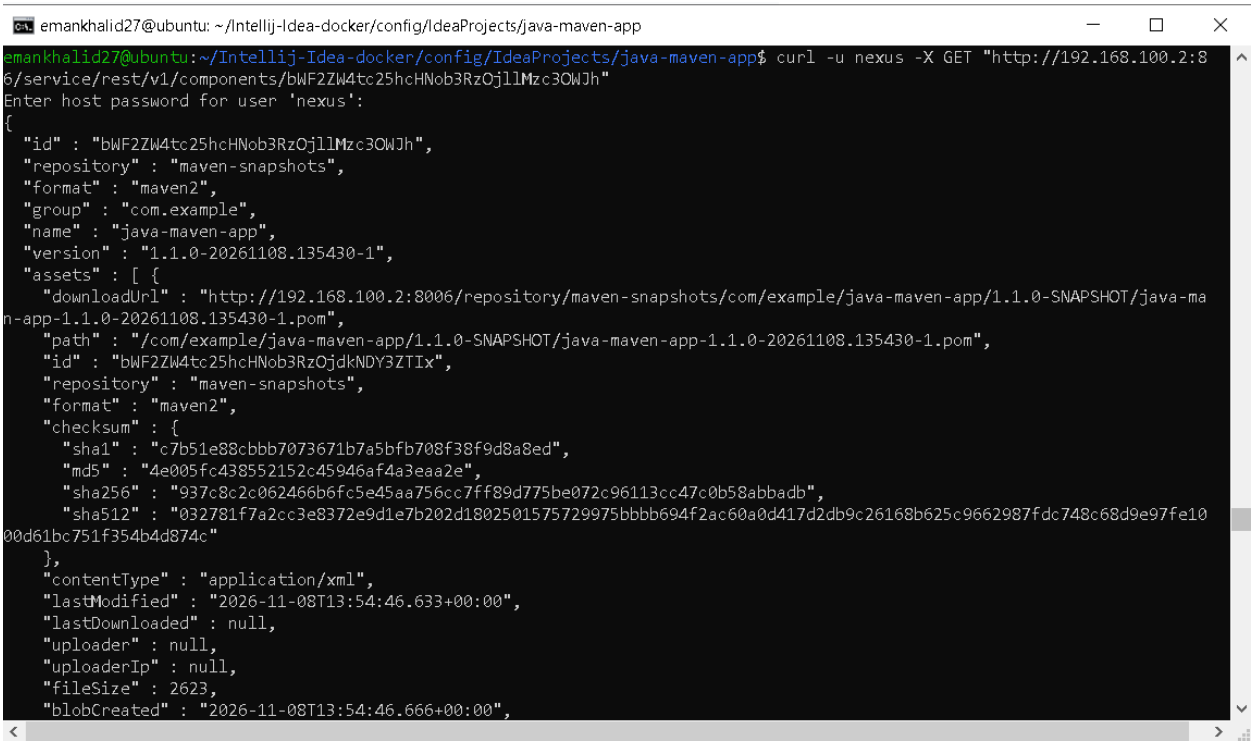


Figure 5.4: Getting detailed information for a specific component by its ID. (nexus_component_details.png)

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

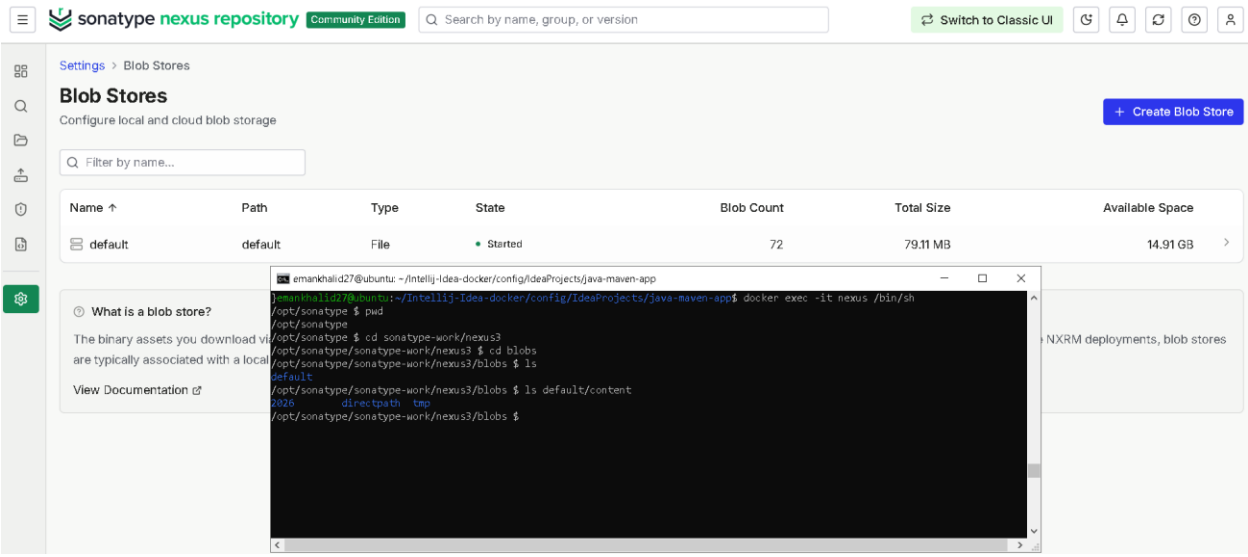


Figure 6.1: Inspecting the default blob-store directory inside the container. (nexus_default_blob_content.png)

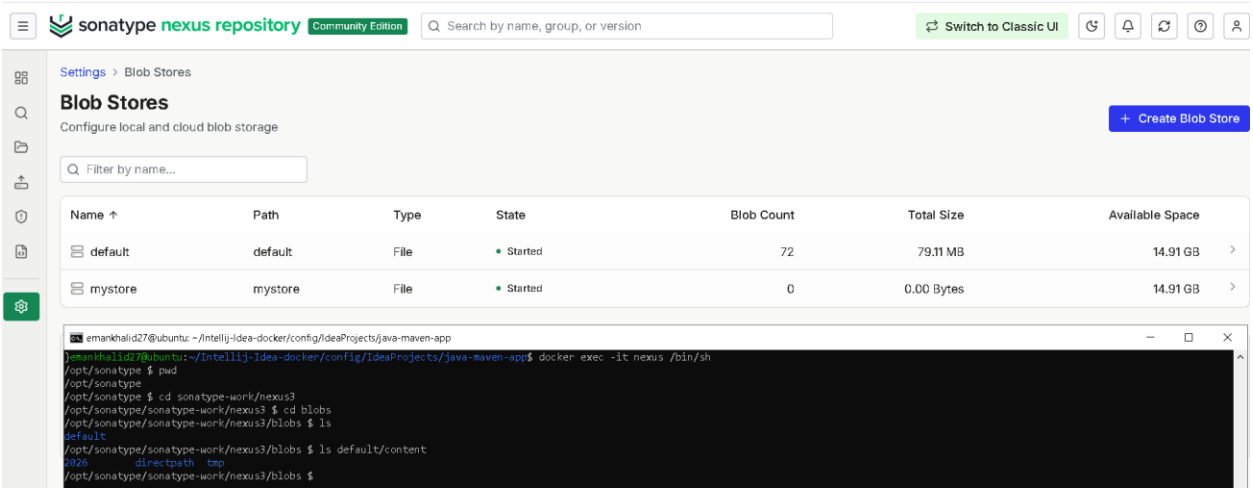


Figure 6.2: The mystore file blob store created and in Started state. (nexus_mystore_created.png)

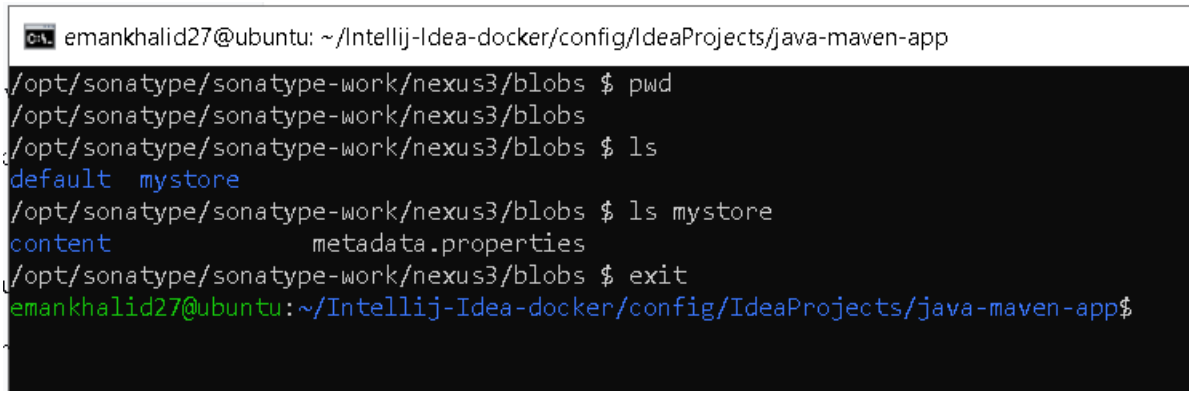


Figure 6.3: Verifying the default and mystore directories inside the container. (nexus_blob_directories.png)

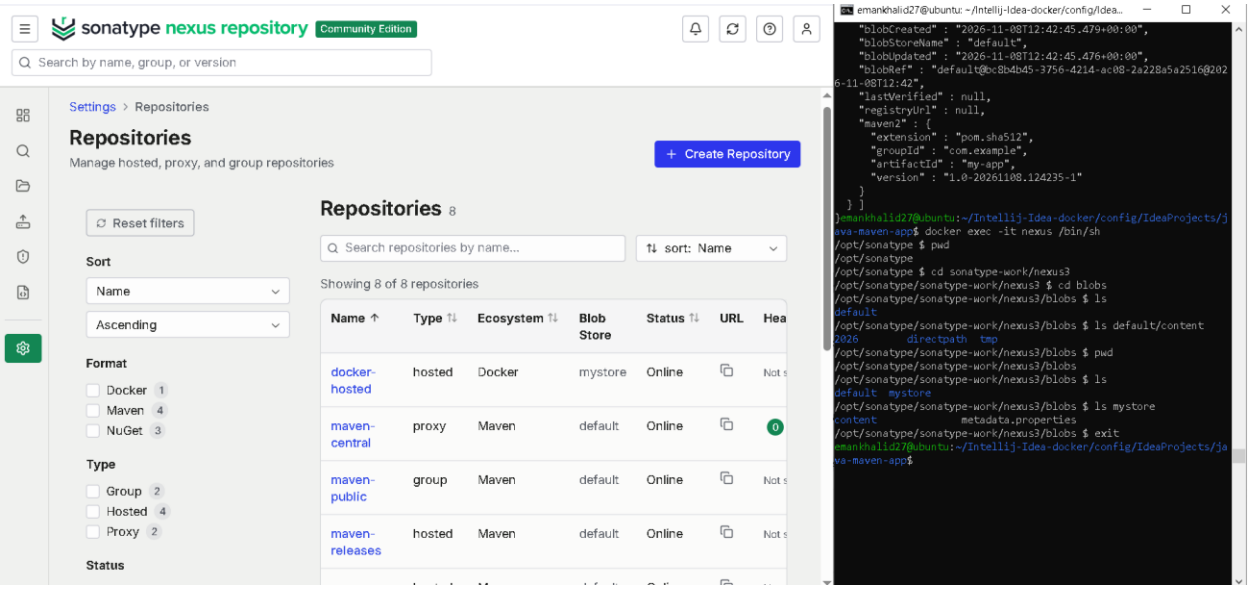


Figure 6.4: The docker-hosted repository created using the mystore blob store. (nexus_docker_hosted_repository.png)

Task 7: Create and Run a Maven Snapshot Cleanup Policy

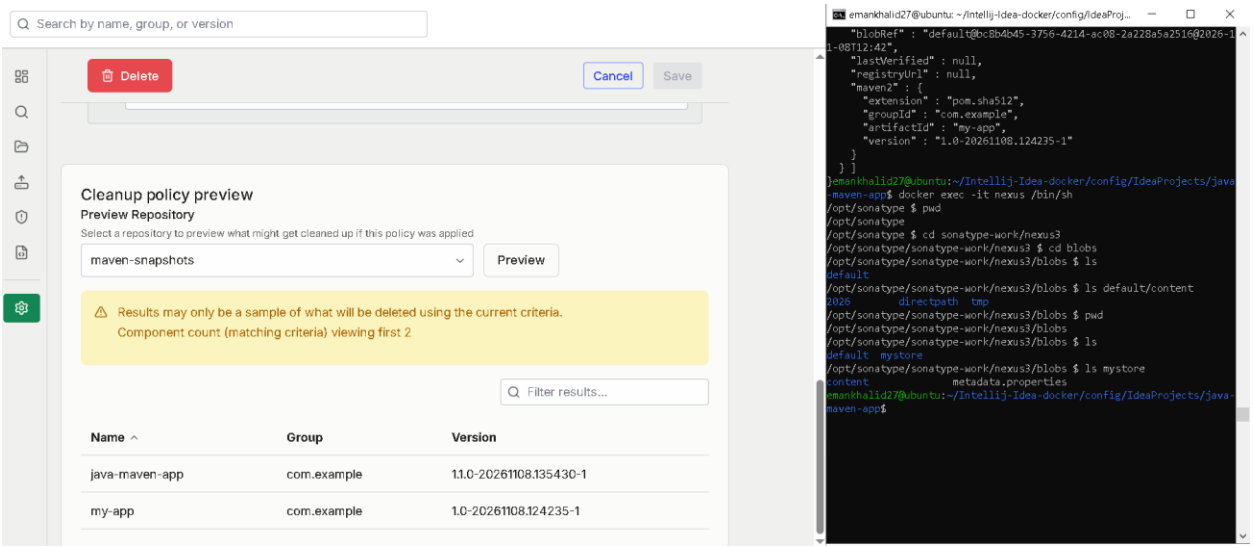


Figure 7.1: The maven cleanup policy with the .* matcher and component preview.
(nexus_cleanup_policy_created.png)

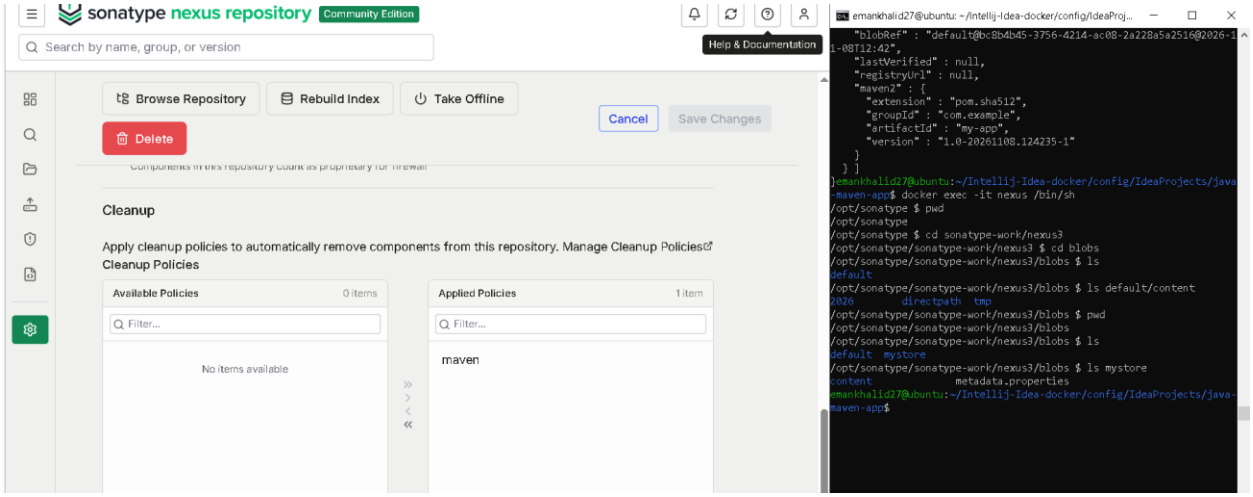


Figure 7.2: The maven policy attached to the maven-snapshots repository.
(nexus_cleanup_policy_attached.png)

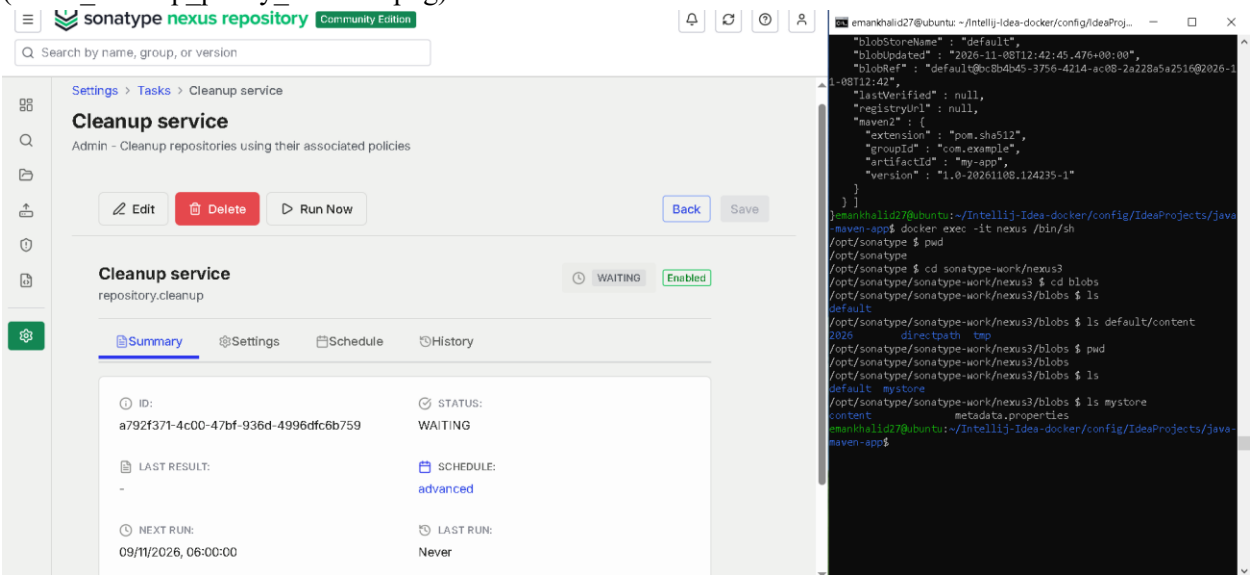


Figure 7.3: The Cleanup service in Waiting state with a 06:00 schedule. (nexus_cleanup_task_waiting.png)

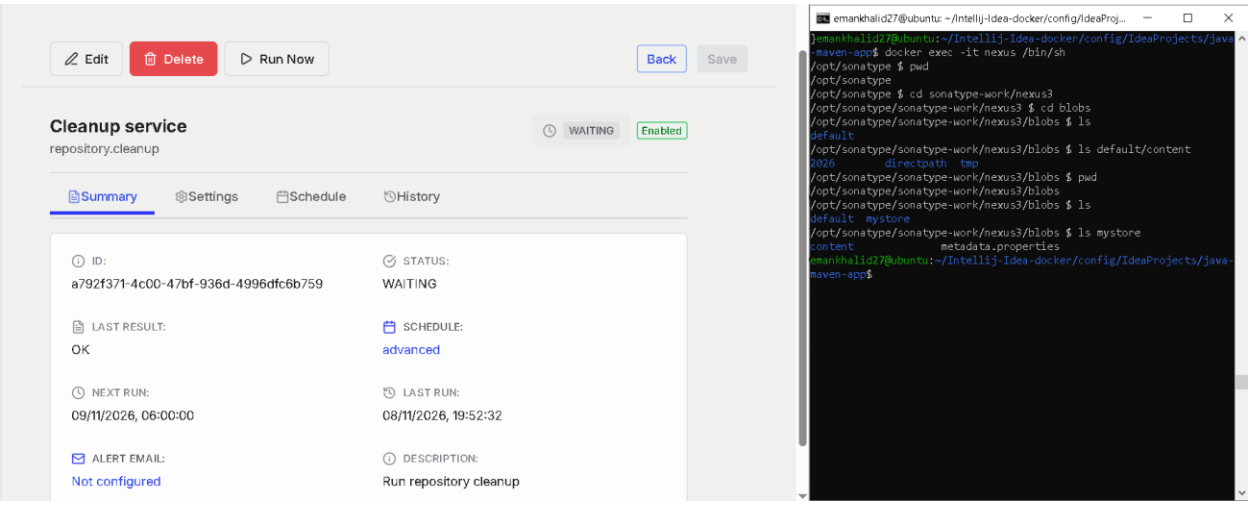


Figure 7.4: The Cleanup service after a manual run showing Last result: Ok. (nexus_cleanup_task_completed.png)

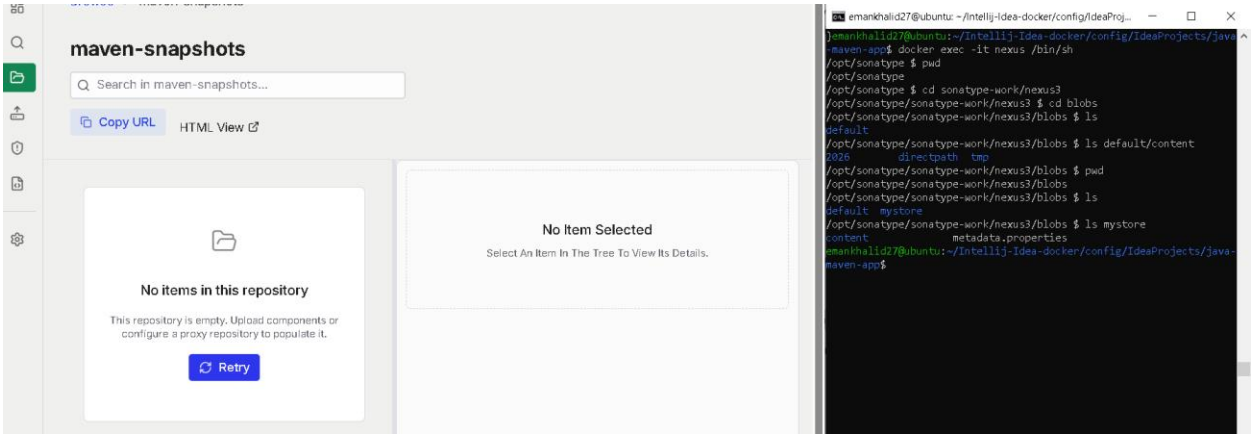


Figure 7.5: The maven-snapshots repository empty after cleanup. (nexus_maven_snapshots_empty.png)

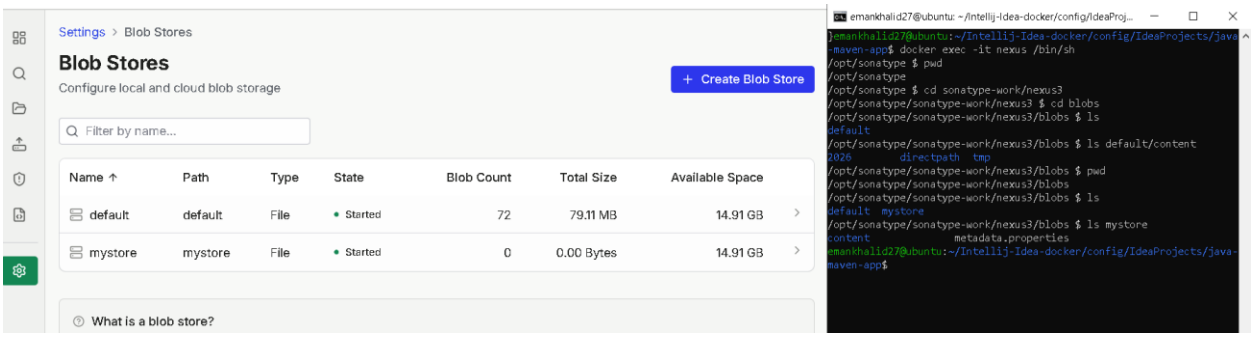


Figure 7.6: The default blob store before compaction. (nexus_blob_store_before_compaction.png)

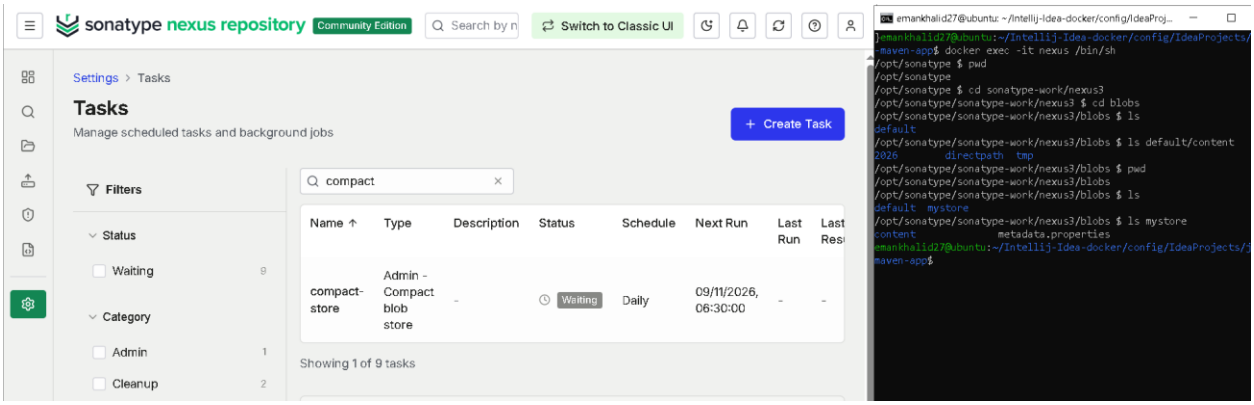


Figure 7.7: The compact-store task configured. (nexus_compact_task_configured.png)

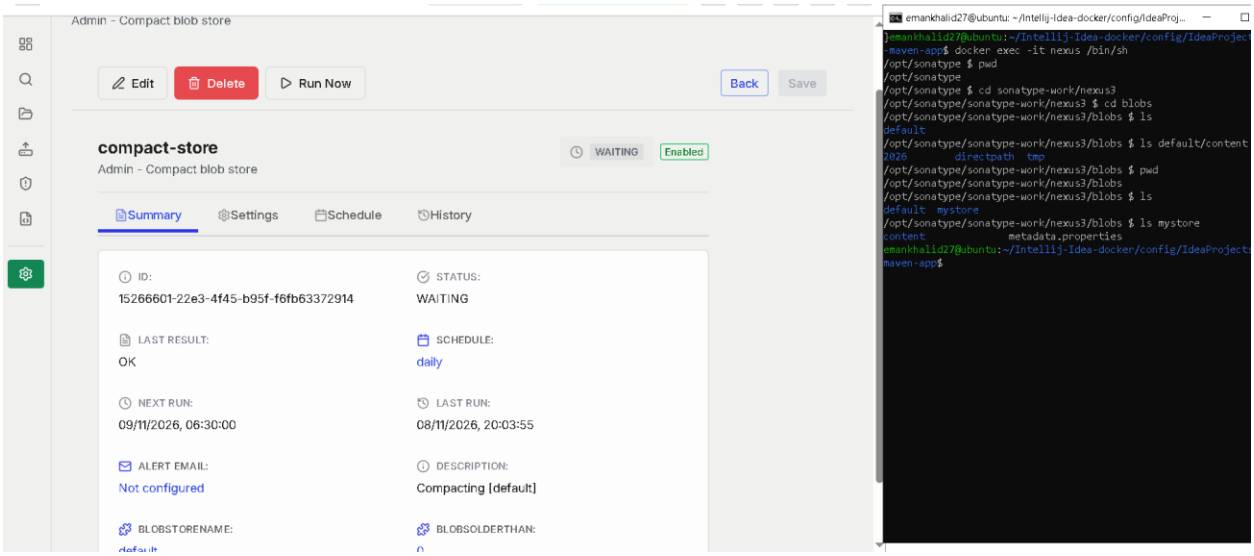


Figure 7.8: The compact-store task after manual execution with Last result: Ok. (nexus_compact_task_completed.png)

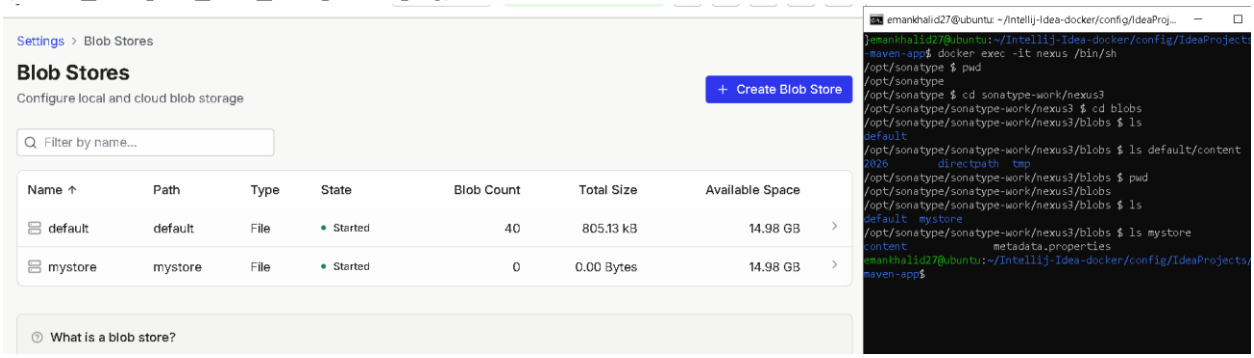


Figure 7.9: The default blob store after compaction showing reclaimed space. (nexus_blob_store_after_compaction.png)

3. Conclusion

I successfully set up a Nexus Repository Manager, configured user access and roles, published artifacts from both Gradle and Maven projects, inspected the repositories via the REST API, created a new blob store and Docker hosted repository, and implemented a cleanup policy with blob store compaction to reclaim disk space. All tasks were completed without exposing any sensitive credentials.